

OpenVLA · arXiv 2406.09246 · CoRL 2024

Stakeholder

18+3

Open HF 7B VLA + LoRA / int4

Teaching + ablation harness — not 2026 BEHAVIOR entrant

Oct 21 paper club · VLA Architectures

Spine: naïve discrete AR @ ~5-6 Hz · historical open AR baseline

BEHAVIOR deploy = $\pi_{0.5}$ / OpenPI / GR00T · OpenVLA = open AR control

Action-family taxonomy · CLUB SPINE

Must appear in every role · no-chunk AR $\approx$ 5-6 Hz

Family	Model	Signature
Naïve discrete AR bins	OpenVLA / RT-2	Lang grounding @ $\sim$ 5-6 Hz; weak high-Hz
DDPM chunks	Octo / DP	Smoother narrow skills
FM chunks	π_0 / $\pi_{0.5}$	2025-26 deploy winners
Compressed discrete AR	FAST / OpenVLA+FAST / π_0 -FAST	AR recovers high-Hz

Syllabus: OpenVLA = historical open AR baseline · BEHAVIOR deploy = $\pi_{0.5}$ / OpenPI / GROOT

Club one-liner

OpenVLA = open Llama/Prismatic discrete-AR VLA on OXE

Peer to Octo diffusion · ancestor contrast to π_0 FM

FAST is the AR patch · not 2026 BEHAVIOR SOTA

Stack: DINOv2||SigLIP + Llama-2 7B @ 224² · 256 bins · no chunk · ~6 Hz

Adoption unlock: LoRA r=32 on 1xA100 · int4 serve ~7 GB

GitHub title cue: [ECE 605] - OpenVLA Making Empiricist

Why OpenVLA still matters · Stakeholder

Open HF 7B

weights · code · notebooks
remote inference server

LoRA on 1×A100

10-15 h · ~1.4% params
68.2 $\approx$ 69.7 full FT

int4 ~7 GB

consumer server
 $\approx$ bf16 Bridge SR

Lang manip FT

multi-object grounding
beats scratch DP

2026 posture

teaching + ablation harness
NOT BEHAVIOR entrant

Ship instead

$\pi_{0.5}$ / OpenPI / GR00T
for deploy

Buy / build decision · no Stakeholder redo later

USE OpenVLA for

- Open VLM→action teaching
- LoRA / int4 recipe reference
- AR-bin ablation control in harness
- Language multi-object FT baseline

DO NOT ship as

- 2026 BEHAVIOR primary controller
- High-Hz / contact-rich default
- Sole stack for long-horizon household
- Replacement for chunked FM/DP

Risks to name

Oversell 2024 WidowX SOTA as frontier · underestimate Hz/chunking gap

Bridge zero-action + RT-2-X 2nd-token API quirks in comparisons

Tattoo board · Empiricist must-cite numbers

Say aloud early · then never lose them

WidowX Bridge

70.6%

vs RT-2-X 50.6 · Octo 20.0

Google robot

85.0%

vs RT-2-X 78.3 · Octo 26.7

Headline

+16.5 pp

vs RT-2-X 55B · 7× smaller

LoRA r=32

68.2%

≈ full FT 69.7 (~1.4% params)

LIBERO FT avg

76.5%

rank 1.5 > Octo 75.1 > DP 72.4

int4 serve

71.9%

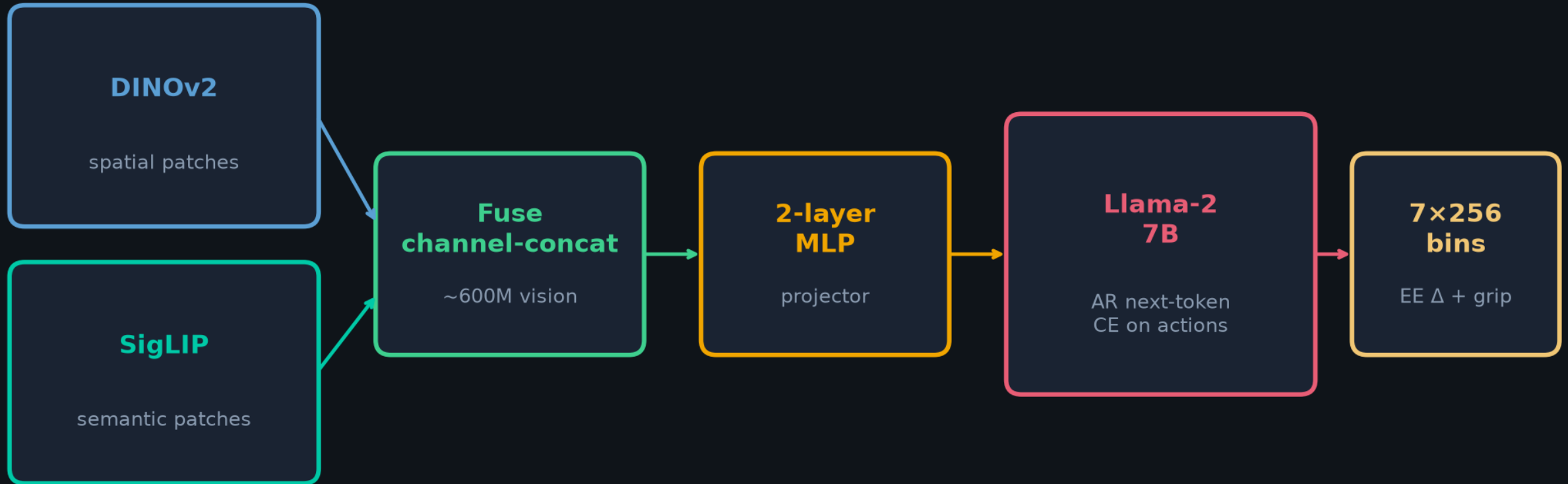
~7.0 GB ≈ bf16 71.3%

Also tattoo: frozen vision 47% · Bridge-only –30 pp · infer ~6 Hz / 4090 bf16

Do NOT claim +16.5 pp or Bridge parity from Partial LIBERO smoke alone

OpenVLA architecture · Prismatic VLM → discrete AR actions

No wrist / proprio / history / chunks · single 3rd-person RGB 224²



Prompt: “What should the robot do to {task}? A:” + 7 action tokens

Train: OXE ~970k · 27 epochs · 64×A100 ×14d ≈ 21.5k A100-h · LR 2e-5 · vision UNFROZEN

PEFT / LoRA table · Table 1 moral

Method	Success	Trainable M	Peak mem
Full FT	69.7 ± 7.2%	7188.1	163.3 GB*
Last layer only	30.3 ± 6.1%	465.1	51.4 GB
Frozen vision	47.0 ± 6.9%	6760.4	156.2 GB*
Sandwich	62.1 ± 7.9%	914.2	64.0 GB
LoRA r=32 ★	68.2 ± 7.5%	97.6 (1.4%)	59.7 GB
LoRA r=64	68.2 ± 7.8%	195.2	60.5 GB

Default r=32 · 10-15 h on 1×A100 · ~8× compute cut vs full FT on 8×A100

Reproduction ladder · Empiricist build order

HF smoke → LoRA r=32 → DP-matched side-by-side → optional FAST+



Scout success = working ckpt + LoRA LIBERO direction + honest Hz/VRAM logs

NOT Bridge 170-rollout parity · env: `/workspace/openvla-quiet/env_outline/`

AR discrete vs DP / FM continuous · Empiricist contrast

Axis	OpenVLA AR	DP / Octo	FM (π_0 family)
Action	7×256 bin ids	Denoise chunk	Flow on chunk
Temporal	Single step NO chunk	Chunk + TE/receding	Chunk + receding
Loss	Token CE	Score / ϵ -pred	FM / velocity
Infer cost	7 AR × 7B LM	K steps × small head	~10 NFE × ~300M
Hz	~6 (4090 bf16)	5-15+ w/ chunks	~30 challenge
Lang	STRONG (VLM NTP)	Needs lang encoder	VLM + task emb
Dexterity	Grid / choppy if slow	STRONG	STRONG
2025-26 role	Open ancestor / lang FT	Strong IL baseline	DEFAULT serve

Pick: open lang FT → OpenVLA LoRA · smooth high-Hz → DP/FM · BEHAVIOR'26 → $\pi_{0.5}$ + optional FAST aux

What to cite OpenVLA for (2026)

CITE FOR

- ✓ Open 7B VLA + LoRA/int4 recipes
- ✓ VLM backbone → discrete actions works
- ✓ Language grounding FT default
- ✓ Ablation morals: mix, vision FT, fused vision
- ✓ Historical open AR baseline in syllabus

DO NOT CITE FOR

- ✗ 2026 BEHAVIOR SOTA control
- ✗ Best high-Hz dexterity
- ✗ Replacement for chunked FM/DP
- ✗ End-to-end multi-cam mobile bimanual
- ✗ Solved generalist manipulation

Novelty is open VLA systems + PEFT/quant — not a new control law

Fig.1 — system card / OpenVLA overview

arXiv:2406.09246v3 [cs.RO] 5 Sep 2024

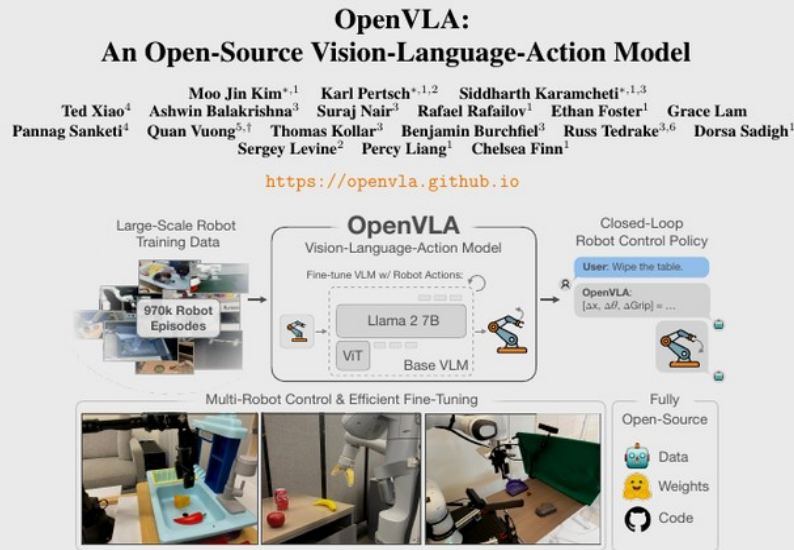


Figure 1: We present OpenVLA, a 7B-parameter open-source vision-language-action model (VLA), trained on 970k robot episodes from the Open X-Embodiment dataset [1]. OpenVLA sets a new state of the art for generalist robot manipulation policies. It supports controlling multiple robots out of the box and can be quickly adapted to new robot domains via parameter-efficient fine-tuning. The OpenVLA checkpoints and PyTorch training pipeline are fully open-source and models can be downloaded and fine-tuned from HuggingFace.

Abstract: Large policies pretrained on a combination of Internet-scale vision-language data and diverse robot demonstrations have the potential to change how we teach robots new skills: rather than training new behaviors from scratch, we can fine-tune such vision-language-action (VLA) models to obtain robust, generalizable policies for visuomotor control. Yet, widespread adoption of VLAs for robotics has been challenging as 1) existing VLAs are largely closed and inaccessible to the public, and 2) prior work fails to explore methods for efficiently fine-tuning VLAs for new tasks, a key component for adoption. Addressing these challenges, we introduce OpenVLA, a 7B-parameter open-source VLA trained on a diverse collection of 970k real-world robot demonstrations. OpenVLA builds on a Llama 2 language model combined with a visual encoder that fuses pretrained features from DINOv2 and SigLIP. As a product of the added data diversity and new model components, OpenVLA demonstrates strong results for generalist manipulation, outperforming closed models such as RT-2-X (55B) by 16.5% in absolute task success rate across 29 tasks and multiple robot embodiments, with 7x fewer parameters. We further show that we can effectively fine-tune OpenVLA for new settings, with especially

^{*}: denotes equal contribution

Correspondence to: moojink@stanford.edu, pertsch@berkeley.edu, skaramcheti@stanford.edu

¹Stanford University, ²UC Berkeley, ³Toyota Research Institute, ⁴Google Deepmind, ⁵Physical Intelligence,

⁶MIT, [†]Work done in part while at Google Deepmind

Fig.3 — WidowX Bridge bars (OpenVLA 70.6%)

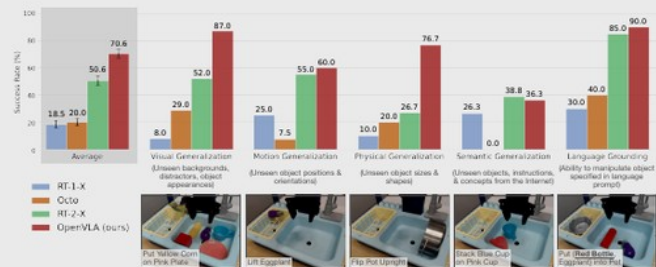


Figure 3: BridgeData V2 WidowX robot evaluation tasks and results. We evaluate OpenVLA and prior state-of-the-art generalist robot policies on a comprehensive suite of tasks covering several axes of generalization, as well as tasks that specifically assess language conditioning ability. OpenVLA achieves highest overall performance and even outperforms closed-source model RT-2-X in all categories except for semantic generalization. Average success rates $\pm$ StdErr are computed across 170 total rollouts per approach. See Table 4 for detailed results.

techniques for large transformer model training such as automatic mixed precision (AMP, PyTorch [75]), FlashAttention [76], and fully sharded data parallelism (FSDP, Zhao et al. [77]). Out of the box, the OpenVLA codebase has full support for training on the Open X dataset, integrates with HuggingFace’s [21] AutoModel class, and supports LoRA fine-tuning [26] and quantized model inference [27, 88].

5 Experiments

The goal of our experimental evaluations is to test OpenVLA’s ability to serve as a powerful multi-robot control policy out of the box, as well as be a good initialization for fine-tuning to new robot tasks. Concretely, we aim to answer the following questions:

- 1. How does OpenVLA compare to prior generalist robot policies, when evaluating on multiple robots and various types of generalization?
- 2. Can OpenVLA be effectively fine-tuned on a new robot setup and task, and how does it compare to state-of-the-art data-efficient imitation learning approaches?
- 3. Can we use parameter-efficient fine-tuning and quantization to reduce the computational requirements for training and inference of OpenVLA models and make them more accessible? What are the performance-compute trade-offs?

5.1 Direct Evaluations on Multiple Robot Platforms

Robot Setups and Tasks. We evaluate OpenVLA’s performance “out-of-the-box” on two robot embodiments: the WidowX robot from the BridgeData V2 evaluations [6] (see Fig. 1, left) and the mobile manipulation robot from the RT-1 and RT-2 evaluations [2, 7] (“Google robot”; see Fig. 1, middle). Both platforms have been extensively used in prior works for evaluating generalist robot policies [1, 2, 5, 7]. We define a comprehensive set of evaluation tasks in each environment that covers various axes of generalization, such as **visual** (unseen backgrounds, distractor objects, colors/appearances of objects); **motion** (unseen object positions/orientations); **physical** (unseen object sizes/shapes); and **semantic** (unseen target objects, instructions, and concepts from the Internet) generalization. We also assess language conditioning ability in scenes with multiple objects, testing whether the policy can manipulate the correct target object, as specified in the user’s prompt. See bottom row of Fig. 3 and Fig. 4 for example task images in the BridgeData V2 and Google robot evaluations, respectively. Overall, we evaluated each method in 170 rollouts (17 tasks with 10 trials each) for BridgeData V2 experiments and 60 rollouts (12 tasks with 5 trials each) for Google robot experiments. A detailed breakdown of all tasks and how they differ from the training data is in

Fig.4 — Google robot bars (OpenVLA 85.0%)

Appendix B. All evaluations in this and the following sections are conducted as A/B evaluations, using the same tasks with the same sets of initial robot and object states, to ensure fair comparison.

Comparisons. We compare OpenVLA’s performance to three prior generalist manipulation policies: **RT-1-X** [1], **RT-2-X** [1], and **Octo** [5]. **RT-1-X** (35M parameters) and **Octo** (93M parameters) are transformer policies trained from scratch on subsets of the OpenX dataset; Octo is the state-of-the-art model among open-source manipulation policies. **RT-2-X** (55B parameters) is a state-of-the-art, closed-source VLA that leverages Internet-pretrained vision and language backbones.

The results are summarized in Fig. 3 for BridgeData V2 evaluations and Fig. 4 for Google robot evaluations (per-task breakdown in Appendix, Table 4 and Table 6). We find that both RT-1-X and Octo struggle on the tested tasks, often failing to manipulate the correct object, especially when distractors are present, and in some cases causing the robot to wave its arm around aimlessly. Note that our evaluations test even larger degrees of generalization than the evaluations performed in those prior works to challenge the Internet-pretrained VLA models. Thus, lower performance of models without Internet pretraining is expected. RT-2-X clearly outperforms both RT-1-X and Octo, demonstrating the benefits of large, pretrained VLMs for robotics.

Notably, OpenVLA performs comparably to RT-2-X on Google robot evaluations and significantly outperforms RT-2-X on BridgeData V2 evaluations despite being an order of magnitude smaller (7B vs. 55B parameters). Qualitatively, we find that both RT-2-X and OpenVLA exhibit markedly more robust behaviors than the other tested models, such as approaching the correct object when distractor objects are present, properly orienting the robot’s end-effector to align with the orientation of the target object, and even recovering from mistakes such as insecurely grasping objects (see <https://openvla.github.io> for qualitative rollout examples). RT-2-X achieves higher performance in semantic generalization tasks, as shown in Fig. 3, which is expected given that it uses larger-scale Internet pretraining data and is co-fine-tuned with both robot action data and Internet pretraining data to better preserve the pretraining knowledge, rather than being fine-tuned solely on robot data, like OpenVLA. However, OpenVLA performs comparably or better in all other task categories in both BridgeData V2 and Google robot evaluations. The performance difference can be attributed to a combination of factors: we curated a much larger training dataset for OpenVLA with 970k trajectories (vs. 350k for RT-2-X); we performed more careful cleaning of the training dataset and, e.g., filtered out all-zero actions in the Bridge dataset (see Appendix C for a detailed discussion); and OpenVLA uses a fused vision encoder that combines pretrained semantic and spatial features. See Appendix D for ablation analyses of these components.

5.2 Data-Efficient Adaptation to New Robot Setups

While prior works mainly focused on directly evaluating VLAs “out-of-the-box” [1, 7, 16], effective fine-tuning of VLA models to new tasks and robot setups is largely unexplored, yet is key for their widespread adoption. In this section, we investigate OpenVLA’s ability to be quickly adapted to a new real-world robot setup. (See Appendix E for fine-tuning experiments in simulation.)

Robot setups and tasks. We test a simple fine-tuning recipe for the OpenVLA model: full fine-tuning of all model parameters, using small datasets with 10–150 demonstrations of a target task (see Fig. 5; we explore parameter-efficient fine-tuning approaches in Section 5.3). We test OpenVLA in two setups: **Franka-Tabletop**, a stationary, table-mounted Franka Emika Panda 7-DoF robot arm; and **Franka-DROID**, the Franka robot arm setup from the recently released DROID dataset [11],

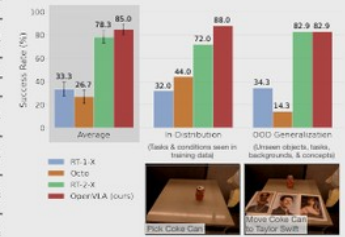


Figure 4: Google robot evaluation results. We evaluate generalist robot policies on in-distribution and out-of-distribution (OOD) tasks on the mobile manipulator used in RT-1 and RT-2 evaluations [2, 7]. We find that OpenVLA and RT-2-X attain comparable performance and significantly outperform RT-1-X and Octo overall. Average success rates $\pm$ StdErr are computed across 60 total rollouts per approach. See Table 6 for detailed results.

Fig.6 — Inference Hz vs GPU

5.3 Parameter-Efficient Fine-Tuning

The full fine-tuning runs of OpenVLA in the previous section used 8 A100 GPUs for 5-15 hours per task (depending on the dataset size) to achieve high performance. While this is substantially less compute than what is required for VLA pretraining, in this section we explore even more compute- and parameter-efficient fine-tuning approaches and investigate their effectiveness.

Concretely, we compare the following fine-tuning approaches: **full fine-tuning** updates all weights during fine-tuning, as described in Section 5.2; **last layer only** fine-tunes only the last layer of OpenVLA’s transformer backbone and the token embedding matrix; **frozen vision** freezes the vision encoder but fine-tunes all other weights; **sandwich fine-tuning** unfreezes the vision encoder, token embedding matrix, and last layer; and **LoRA** uses the popular low-rank adaptation technique of Hu et al. [26] with multiple rank values r , applied to all linear layers of the model.

We report fine-tuning success rates across multiple Franka-Tabletop tasks, as well as training parameter count and GPU memory requirements, in Table 1.⁴ We find that only fine-tuning the network’s last layer or freezing the vision encoder leads to poor performance, suggesting that further adaptation of the visual features to the target scene is crucial. In contrast, “sandwich fine-tuning” achieves better performance since it fine-tunes the vision encoder, and it consumes less GPU memory since it does not fine-tune the full LLM backbone. Lastly, LoRA achieves the best trade-off between performance and training memory consumption, outperforming “sandwich fine-tuning” and matching full fine-tuning performance while fine-tuning only 1.4% of the parameters. We find that the LoRA rank has negligible effect on policy performance and thus recommend using a default rank of $r = 32$. With LoRA, we can fine-tune OpenVLA on a new task within 10-15 hours on a *single* A100 GPU – an 8x reduction in compute compared to full fine-tuning.

5.4 Memory-Efficient Inference via Quantization

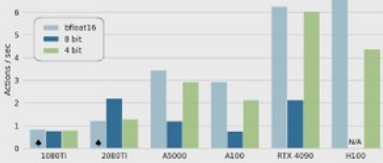


Figure 6: **OpenVLA inference speed for various GPUs.** Both bfloat16 and int4 quantization achieve high throughput, especially on GPUs with Ada Lovelace architecture (RTX 4090, H100). Further speed-ups are possible with modern LLM inference frameworks like TensorRT-LLM [89]. $\blacktriangle$: Model sharded across two GPUs to fit.

OpenVLA, a 7B-parameter model, consumes more memory at inference time than prior open-source generalist policies such as Octo, which has $<100\text{M}$ parameters. We follow best-practices from LLM serving by saving and loading OpenVLA in bfloat16 precision for inference (our default approach), which cuts the memory footprint in half, allowing us to serve OpenVLA on GPUs with only 16GB

Table 1: **Parameter-efficient fine-tuning evaluation.** LoRA fine-tuning achieves the best performance-compute trade-off, matching full fine-tuning performance while training only 1.4% of the model parameters. Mean success $\pm$ StdErr computed across 33 rollouts per approach on select Franka-Tabletop tasks (see Table 8 for details). *: Sharded across 2 GPUs with FSDP [77].

Strategy	Success Rate	Train Params ($\times 10^9$)	VRAM (batch 16)
Full FT	69.7 ± 7.2 %	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	68.2 ± 7.5 %	97.6	59.7 GB
rank=64	68.2 ± 7.8 %	195.2	60.5 GB

Precision	Bridge Success	VRAM
bfloat16	71.3 ± 4.8 %	16.8 GB
int8	58.1 ± 5.1 %	10.2 GB
int4	71.9 ± 4.7 %	7.0 GB

Table 2: **Performance with quantized inference.** 4-bit quantization matches the performance of bfloat16 inference (our default approach) while reducing the GPU memory footprint by more than half. Mean success $\pm$ StdErr computed across 8 representative BridgeData V2 tasks [6] and 80 rollouts per approach (see Table 5 for details).

⁴In Section 5.3 and Section 5.4, we experiment with a version of the OpenVLA model that is pretrained with a smaller robot data mixture (the same OpenX dataset mixture as Octo) and has a slightly smaller architecture which only uses a SigLIP [79] vision backbone instead of the fused DinoSigLIP encoder. We find that this simpler architecture still achieves strong performance in both fine-tuning tasks and “out-of-the-box” tasks.

Harness vs deploy · Visionary → 2026 BEHAVIOR

Skip Discord / Drive entirely

HARNESS · keep

- OpenVLA as bin-AR control
- LoRA / int4 recipes
- Language multi-object evals
- OXE-style mix morals
- Demo cleaning (no-ops!)

DEPLOY · ship

- $\pi_{0.5}$ / OpenPI / GR00T-class
- Chunked continuous FM
- Optional FAST aux (not primary)
- Multi-cam + proprio + stages
- Task/stage tokens for fixed-50

1 Do not ship pure OpenVLA AR as primary 30 Hz controller

2 If AR research continues → FAST/chunks before high-Hz claims

3 Steal LoRA recipe into FM stacks

4 Honesty: Bridge/Google % $\neq$ BEHAVIOR q-score

Stakeholder · close

Open HF + LoRA recipes stay. Ship $\pi_{0.5}$ /GR00T for BEHAVIOR.

Teaching + ablation harness — not 2026 entrant

Q&A · Oct 21 · OpenVLA / 2406.09246

Buy / build decision · no Stakeholder redo later

USE OpenVLA for

- Open VLM→action teaching
- LoRA / int4 recipe reference
- AR-bin ablation control in harness
- Language multi-object FT baseline

DO NOT ship as

- 2026 BEHAVIOR primary controller
- High-Hz / contact-rich default
- Sole stack for long-horizon household
- Replacement for chunked FM/DP

Risks to name

Oversell 2024 WidowX SOTA as frontier · underestimate Hz/chunking gap

Bridge zero-action + RT-2-X 2nd-token API quirks in comparisons